

Lab 5: Implementing the Microservice (Implementing the Leave Microservice)

This lab focuses on the practical implementation of one of the core services defined in Lab 4: the **Leave Service**. This service will operate completely independently, owning its own database (Leave Request data) and providing Service Contracts via a RESTful API, strictly adhering to the principles of Microservices Architecture.

Objectives

1. Set up a standalone Node.js/Express application dedicated solely to Leave Management operations.
2. Implement the Leave Service logic and connect to the database (PostgreSQL).
3. Expose the Service Contracts (REST API Endpoints) to read leave balance data and search for leave requests.
4. Test the service independently (Isolation Testing).

Tool & Technology Setup

We will use Node.js and Express, combined with the pg library (PostgreSQL Client) to access data from the designed Database.

Tool	Purpose	Installation/Setup Guide
Node.js (v16+)	Core programming language runtime environment.	Ensure Node.js is installed on your machine.
Express.js	Lightweight web framework for building the Application/API Layer.	Run command: npm install express
pg (node-postgres)	Library for connecting and executing SQL directly to PostgreSQL.	Run command: npm install pg dotenv
Postman / cURL	API Testing tools.	Install Postman or use the curl command in the Terminal.

Activity Practice 1: Project Setup and Data Connection

Goal: Create the project directory structure and set up the PostgreSQL database connection for the Microservice.

Step-by-Step Instructions & Coding Guide:

1. Create Service Directory:

bash

Show full code block

Ensure you are in the root directory of the project

```
mkdir leave_service_lab
```

```
cd leave_service_lab
```

```
npm init -y
```

```
npm install express pg dotenv
```

```
touch app.js db.js .env
```

2. Configure Environment Variables (.env): Contains Database connection information. File: .env

env

Show full code block

```
PORT=5002
```

```
DB_USER=postgres
```

```
DB_PASSWORD=yourpassword
```

```
DB_HOST=localhost
```

```
DB_PORT=5432
```

```
DB_NAME=hrm_db
```

3. Initialize PostgreSQL Connection: Use a Connection Pool to manage connections efficiently. File: [db.js](#)

javascript

Show full code block

```
const { Pool } = require("pg");
```

```

require("dotenv").config();

// Configure connection to PostgreSQL specifically for the Leave Service
const pool = new Pool({
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  database: process.env.DB_NAME,
});

pool.connect()
  .then(() => console.log("✅ Successfully connected to the Database!"))
  .catch(err => console.error("❌ Database connection error", err));

module.exports = pool;

```

Activity Practice 2: Implementing the Service API

Goal: Implement the REST API Endpoints to read leave data, fulfilling the Service Contract defined in Lab 4.

Step-by-Step Instructions & Coding Guide:

1. **Initialize Express and the Request List Endpoint (GET /api/leaves):** Allows retrieving all requests, with support for status filtering. File: app.js

javascript

Show full code block

```

const express = require("express");
const pool = require("./db");

```

```
require("dotenv").config();

const app = express();
app.use(express.json());

// API 1: Get the list of leave requests (Supports filtering)
app.get('/api/leaves', async (req, res) => {
  try {
    const statusFilter = req.query.status;
    let query = "SELECT * FROM leave_requests WHERE 1=1";
    let params = [];

    if (statusFilter) {
      query += " AND status = $1";
      params.push(statusFilter);
    }

    query += " ORDER BY created_at DESC";

    const result = await pool.query(query, params);
    res.status(200).json(result.rows);
  } catch (err) {
    console.error(err);
    res.status(500).json({ message: "Internal Server Error" });
  }
});
```

2. Implement the View Leave Balance Endpoint (GET

/api/leaves/balance/:user_id): Allows retrieving the maximum leave days and used leave days of an employee. File: app.js (Add below API 1)

javascript

Show full code block

// API 2: View the leave balance of a specific employee

```
app.get('/api/leaves/balance/:user_id', async (req, res) => {
```

```
  const { user_id } = req.params;
```

```
  const currentYear = new Date().getFullYear();
```

```
  try {
```

```
    // Get the maximum leave days (Communicate with the users table to read static info)
```

```
    const userRes = await pool.query("SELECT max_leave_days FROM users WHERE id = $1", [user_id]);
```

```
    if (userRes.rows.length === 0) {
```

```
      return res.status(404).json({ message: "Employee not found" });
```

```
    }
```

```
    const maxDays = userRes.rows[0].max_leave_days || 12;
```

```
    // Calculate the total number of days taken in the current year
```

```
    const usedRes = await pool.query(`
```

```
      SELECT SUM(total_days) as used
```

```
      FROM leave_requests
```

```
      WHERE user_id = $1 AND status = 'APPROVED'
```

```
      AND EXTRACT(YEAR FROM start_date) = $2
```

```
    `, [user_id, currentYear]);
```

```

const usedDays = parseInt(usedRes.rows[0].used) || 0;

res.status(200).json({
  user_id: parseInt(user_id),
  max_days: maxDays,
  used_days: usedDays,
  remaining_days: maxDays - usedDays
});
} catch (err) {
  console.error(err);
  res.status(500).json({ message: "Error retrieving leave balance" });
}
});

```

3. **Run the Microservice:** Ensure the service runs on a dedicated port (e.g., 5002) to avoid conflicts with other services (Auth, Salary). File: app.js (Add to the end of the file)

javascript

Show full code block

```

const PORT = process.env.PORT || 5002;

if (require.main === module) {
  app.listen(PORT, () => {
    console.log(`🚀 Leave Microservice is running on port ${PORT}`);
  });
}

```

Activity Practice 3: Isolation Testing

Goal: Verify that the service operates correctly and is completely independent of the rest of the system.

Step-by-Step Instructions:

1. Start the Service:

bash

Ensure you are in the leave_service_lab directory

node app.js

(You should see the text: ☒ Successfully connected to the Database! and  Leave Microservice is running on port 5002)

2. Test Fetching the Request List (Using cURL or Postman):

- Action: Send an HTTP GET request to fetch all leave requests.
- cURL Command: `curl -X GET http://127.0.0.1:5002/api/leaves`
- Expected Result: A JSON array containing the leave requests seeded in the database (HTTP 200 OK).

3. Test the Filter Feature:

- Action: Only fetch pending requests (PENDING).
- cURL Command: `curl -X GET "http://127.0.0.1:5002/api/leaves?status=PENDING"`
- Expected Result: A JSON array containing only the requests of employees taking sick leave and sudden personal leave.

4. Test Fetching Leave Balance:

- Action: Query the leave balance of the employee with ID 4 (Hank Nguyen).
- cURL Command: `curl -X GET http://127.0.0.1:5002/api/leaves/balance/4`
- Expected Result: A detailed JSON object: `{"user_id": 4, "max_days": 12, "used_days": 1, "remaining_days": 11}` (HTTP 200 OK).

5. Test Error Handling:

- Action: Query a non-existent employee ID (e.g., ID 999).

- cURL Command: `curl -X GET http://127.0.0.1:5002/api/leaves/balance/999`
- Expected Result: Returns JSON `{"message": "Employee not found"}` with an HTTP 404 Not Found error code.